

Thread Tabanlı Convex (Dışbükey) Poligon Kontrolü

Çapraz Çarpım Tabanlı Konvekslik Testinin Java İş Parçacıkları ile Paralleleştirilmesi

Duha Keskin (22360859003) Yunus Alim Avşar (23360859710)

Paralel Programlama — Haziran 2026

<https://github.com/YunusAlim/paralelProgramlama>

Abstract

Düzlemde sıralı (x, y) koordinatlarından oluşan bir nokta dizisinin kapalı bir convex poligon oluşturup oluşturmadığını saptayan, Java `java.util.concurrent` altyapısıyla paralelleştirilmiş bir program sunulmaktadır. Konvekslik, ardışık üçlülerin çapraz çarpım *işareti* ile birlikte işaretli dış açılarının toplamının $\pm 2\pi$ olması koşuluyla *tam-doğru* biçimde test edilir; bu sayede pentagram gibi kendiyle kesişen poligonlar da doğru reddedilir. İş yükü, paylaşılan bir `ExecutorService` havuzu üzerinde çekirdek sayısı kadar parçaya bölünür ve sonuçlar kilit kullanılmadan bir *reduction* ile birleştirilir. Apple M3 Pro (12 çekirdek) üzerinde, $N = 10^7$ nokta için **6.33**× hızlanma ve thread sayısına göre $E = S/C$ verimliliğinin performans çekirdeği sınırında düşüşü ölçülmüş; sonuçlar Amdahl Yasası ve Karp-Flatt seri-pay metriğiyle yorumlanmıştır.

1 Problemin Tanımı

Sıralı (x_i, y_i) , $i = 0, \dots, n-1$ köşelerinden oluşan kapalı bir poligonun **convex** (dışbükey) olup olmadığını belirlemek istiyoruz. Çözüm hem **tek thread (seri)** hem de **çok thread (paralel)** olarak gerçekleştirildi; iki sürümün süreleri ölçülerek hızlanma katsayısı hesaplandı. Problem, çarpışma denetimi, robot hareket planlama ve CBS gibi alanlarda temel bir geometrik yordamdır ve birazdan görüleceği gibi *veri-paralel* bir yapıya sahiptir.

2 Algoritma ve Çalışma Prensibi

2.1 Çapraz çarpım ve dönüş yönü

Ardışık üç köşe P_i, P_{i+1}, P_{i+2} için dönüş yönü, gelen kenar $\vec{e}_i = P_{i+1} - P_i$ ile giden kenar $\vec{e}_{i+1} = P_{i+2} - P_{i+1}$ çapraz çarpımının z bileşeni (bir 2×2 determinant) ile bulunur:

$$z_i = \det \begin{pmatrix} x_{i+1} - x_i & x_{i+2} - x_{i+1} \\ y_{i+1} - y_i & y_{i+2} - y_{i+1} \end{pmatrix} = (x_{i+1} - x_i)(y_{i+2} - y_{i+1}) - (y_{i+1} - y_i)(x_{i+2} - x_{i+1}). \quad (1)$$

$$\text{dönüş}(P_{i+1}) = \begin{cases} \text{sola (saat yönü tersi)} & z_i > 0, \\ \text{sağa (saat yönü)} & z_i < 0, \\ \text{doğrusal (collinear)} & z_i = 0. \end{cases} \quad (2)$$

İndisler mod n alınır; böylece kapamış üçlülere (P_{n-1}, P_0, P_1) ve (P_{n-2}, P_{n-1}, P_0) de denetlenir.

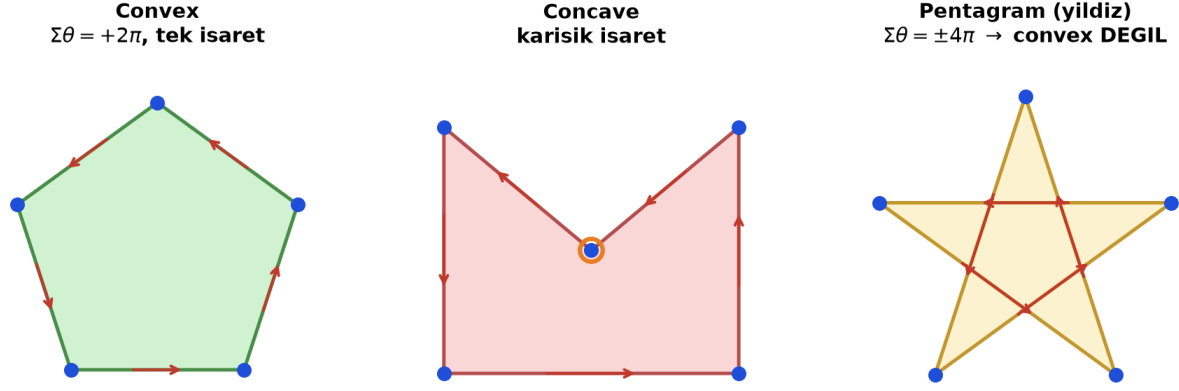
2.2 Gerekli ve yeterli koşul

“Tüm z_i aynı işaretli” koşulu konvekslik için **gerekli ama yeterli değildir**: Şekil 1’teki pentagram (yıldız) tüm dönüşleri aynı yönde olduğu hâlde kendiyle kesişir ve convex değildir. Yeterlilik için poligonun sınırının *tam bir kez* dolandığını, yani işaretli dış (dönüş) açılarının toplamının $\pm 2\pi$ olduğunu da doğrularız:

$$\theta_i = \text{atan2}(z_i, \vec{e}_i \cdot \vec{e}_{i+1}), \quad \text{convex} \iff \underbrace{(\text{sgn } z_i \text{ sabit})}_{\text{tek dönüş yönü}} \wedge \underbrace{\left| \sum_{i=0}^{n-1} \theta_i \right| = 2\pi}_{\text{tek tur}}. \quad (3)$$

Buna göre basit convex poligonda toplam $\pm 2\pi$, pentagramda $\pm 4\pi$, 180° geri-dönüş (spike) veya tüm-doğrusal dejenere girdilerde ise 2π ’den sapar ve doğru biçimde reddedilir. Açı toplamı $\sum \theta_i$ bir *toplama* olduğundan, çapraz çarpım işareti ile birlikte veri-paralel olarak hesaplanabilir.

Tüm donusler aynı yönde olsa bile convex olmayabilir: yeterlilik için $|\Sigma\theta| = 2\pi$ gerekir



Şekil 1: Üç örnek: (sol) convex beşgen $\Sigma\theta = +2\pi$; (orta) concave “V” örneği, karışık işaret; (sağ) pentagram, tüm dönüşler aynı yönde ama $\Sigma\theta = \pm 4\pi$ ve kendini kesişen $\Rightarrow$ convex değil.

2.3 Sınıflar ve fonksiyonlar

Proje OOP ilkeleriyle dört dosyaya ayrıldı:

- `Point` — salt veri sınıfı (x, y çiftini tutar).
- `ChunkResult` — bir alt-aralığın reduction sonucu: (sign, turnSum, mixed).
- `ConvexCheckerTask` implements `Callable<ChunkResult>` — verilen $[start, end)$ aralığındaki üçlüler için çapraz çarpım işaretini ve işaretli açı toplamını tek geçişte üretir.
- `Main` — veri üretimi, paylaşılan havuz, ölçüm, `decide()` birleştirme ve CSV çıktısı.

2.4 Çalışma örnekleri

`generateConvexPolygon(5)`, yarıçapı 100 olan çember üzerinde $(100, 0), (30.9, 95.1), (-80.9, 58.8), (-80.9, -58.8), (30.9, -95.1)$ noktalarını üretir; tüm $z_i > 0$ ve $\Sigma\theta = 2\pi \Rightarrow$ **convex**. `makeConcaveSample()` ise $(0, 0), (4, 0), (4, 4), (2, 2), (0, 4)$ döndürür; $(4, 4) \rightarrow (2, 2) \rightarrow (0, 4)$ üçlüsü ters döner (karışık işaret) $\Rightarrow$ **convex değil**. `makePentagram()` beş çember noktasını yıldız sırasında dolaşır; tek işaret ama $\Sigma\theta = 4\pi \Rightarrow$ **convex değil**. Tüm bu durumlar programın doğruluk testlerinde otomatik denetlenir.

3 Paralel Çözüm

3.1 İşin bölünmesi (data decomposition)

N noktalık dizi, $C = \text{Runtime.getRuntime().availableProcessors()}$ adet çekirdeğe yaklaşık eşit chunk'lara ayrılır:

$$\text{chunk}_k = [k \lfloor N/C \rfloor, (k+1) \lfloor N/C \rfloor), k = 0, \dots, C-2; \quad \text{chunk}_{C-1} = [(C-1) \lfloor N/C \rfloor, N). \quad (4)$$

Son chunk kalan elemanları sahiplenir; hiçbir köşe atlanmaz veya iki kez sayılmaz. İndis i 'ye ait dönüş tam olarak bir chunk'a düştüğü için N köşenin tamamı *tam bir kez* işlenir.

3.2 Thread havuzu ve kilitsiz birleştirme

`new Thread()` yerine `tek bir Executors.newFixedThreadPool` havuzu programda bir kez kurulur ve *tüm* ölçümlerde yeniden kullanılır; böylece havuz kurulum/yıkım maliyeti ölçülen pencerenin dışında kalır (bkz. §4). Her görev `submit()` ile gönderilir, `Future.get()` örtük senkronizasyon sağlar. Thread'ler **paylaşılan bir değişkene yazmaz**; her biri yerel sayaç ve yerel açı toplamı tutup tek bir `ChunkResult` döndürür. Bu yüzden `synchronized`, `Lock` veya `Atomic*` gibi primitiflere gerek kalmaz; kilit çekişmesi sıfırdır. Nihai karar (Denklemler 3) `Main.decide()` içinde bir *reduction* ile verilir: işaret tutarlılığı kontrol edilir ve kısmî açı toplamaları $\Sigma\theta$ 'ya eklenir.

Algoritma 1: Paralel Konvekslik Kontrolü (decomposition + reduction)

Input: Noktalar $P = \langle P_0, \dots, P_{N-1} \rangle$; çekirdek sayısı C ; paylaşılan havuz `pool`

Output: poligon convex mi? (doğru/yanlış)

```
1 if  $N < 3$  then
2   return hata (en az 3 nokta gerekir)
3 futures  $\leftarrow$  boş liste
4 for  $k \leftarrow 0$  to  $C - 1$  do
5    $(\text{start}, \text{end}) \leftarrow \text{chunk}_k$ 
6   futures'a pool.submit(CHUNKSCAN(P, start, end)) ekle
7  $R \leftarrow$  tüm future.get() sonuçları // örtük senkronizasyon
8 return DECIDE(R)

9 fonksiyon CHUNKSCAN(P, start, end):
10    $\text{sign} \leftarrow 0$ ;  $\tau \leftarrow 0$ 
11   for  $i \leftarrow \text{start}$  to  $\text{end} - 1$  do
12      $z \leftarrow$  çapraz çarpım;  $\tau \leftarrow \tau + \text{atan2}(z, \vec{e}_i \cdot \vec{e}_{i+1})$ 
13     işaret tutarlılığını güncelle; karışık işaret görülürse mixed
14   return  $(\text{sign}, \tau, \text{mixed})$ 

15 fonksiyon DECIDE(R):
16    $\Sigma\tau \leftarrow \sum_{r \in R} r.\tau$ 
17   if mixed var / işaretler çelişir / hiç işaret yok then return yanlış
18   return  $||\Sigma\tau| - 2\pi| < \varepsilon$ 
```

Klasik tuzak. Her thread'in yalnızca kendi chunk'ı içinde "convex mi?" kararı vermesi hatalıdır: A chunk'ı hep sola, B chunk'ı hep sağa dönerse ikisi de bağımsızca "convex" der. Bu projede thread'ler yalnızca *işaret + açı* döndürür; küresel kararı `decide()` verir.

4 Deneysel Kurulum

Bileşen	Değer
İşlemci	Apple M3 Pro — 12 çekirdek (6 performans + 6 verim), tek soket
Bellek / OS	18 GB birleşik bellek; macOS 26.4.1 (arm64)
JDK / JVM	OpenJDK 21.0.10 (JetBrains Runtime), <code>-Xmx4g</code>
Zamanlama	<code>System.nanoTime()</code> (monotonik, ns çözünürlük)
Yöntem	boyut başına 10 ısınma + 15 ölçüm; medyan + min + std
DCE koruması	sonuç <i>volatile</i> bir sink'e işlenir (JIT ölü-kod elemesi engellenir)

Tablo 1: Deney ortamı ve ölçüm metodolojisi. Tüm sayılar tek bir koşunun `results.csv/results_threads.csv` çıktısından alınmıştır; ondalık ayraç noktadır.

JVM'in dinamik JIT derleyicisi nedeniyle her boyut için önce 10 ısınma turu (zamanlanmaz) koşulur; ardından 15 ölçümün **medyanı** alınır (tek bir GC duraklamasına karşı ortalamadan daha dayanıklıdır). Sonuçlar bir sink'te tüketildiği için derleyici döngüyü ölü-kod olarak eleyemez. `generateConvexPolygon` maliyeti zamanlanan pencerenin dışındadır ve 10^7 'lik poligon yalnızca bir kez üretilir.

5 Deneysel Sonuçlar

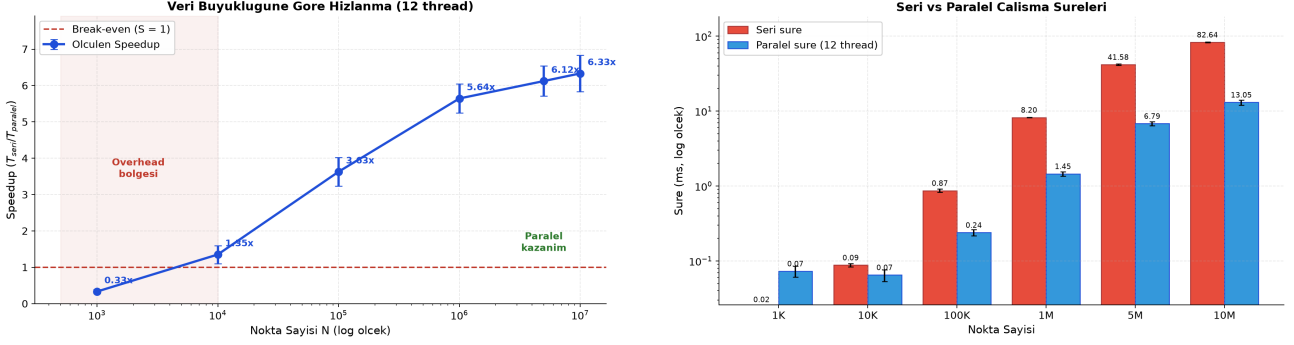
Hızlanma ve verimlilik:

$$S = \frac{T_{\text{seri}}}{T_{\text{paralel}}}, \quad E = \frac{S}{C}. \quad (5)$$

Hangi parametrede ne kadar hızlanma? Sabit $C = 12$ için $N=10^4 \rightarrow 1.35\times$, $N=10^5 \rightarrow 3.63\times$, $N=10^6 \rightarrow 5.64\times$, $N=10^7 \rightarrow 6.33\times$. Sabit $N=10^7$ için $C=2 \rightarrow 1.99\times$, $C=4 \rightarrow 3.86\times$, $C=8 \rightarrow 4.84\times$, $C=12 \rightarrow 6.43\times$.

N	T_{seri} (ms)	T_{paralel} (ms)	S	$E = S/C$	Convex?
1 000	0.024	0.073	$0.33\times$	%3	evet
10 000	0.088	0.065	$1.35\times$	%11	evet
100 000	0.867	0.239	$3.63\times$	%30	evet
1 000 000	8.198	1.453	$5.64\times$	%47	evet
5 000 000	41.58	6.79	$6.12\times$	%51	evet
10 000 000	82.64	13.05	$6.33\times$	%53	evet

Tablo 2: Veri büyüklüğüne göre süreler, hızlanma ($C = 12$ thread) ve verimlilik.



Şekil 2: (sol) N 'ye göre Speedup (yarı-log, hata çubuklu): kırılma noktası $N = 10^4$. (sağ) Seri vs paralel süreler (log- y).

6 Analiz

Küçük N ve kırılma noktası. $N \leq 10^3$ için $S < 1$: `submit/get` koordinasyonunun sabit maliyeti, küçük iş yükünden büyüktür. Havuz *yeniden kullanıldığı* için bu maliyet yalnızca koordinasyondur (havuz kurulumu değil) ve kırılma noktası daha $N = 10^4$ 'te aşılır; bu noktadan sonra paralel sürüm üstünlüğe geçer (Şekil 2).

Amdahl ve Karp–Flatt. Programın seri kalan payı s ise teorik üst sınır

$$S_{\max}(C) = \frac{1}{s + \frac{1-s}{C}}. \quad (6)$$

Seri payı veriden kestirmek için Karp–Flatt metriği kullanılır:

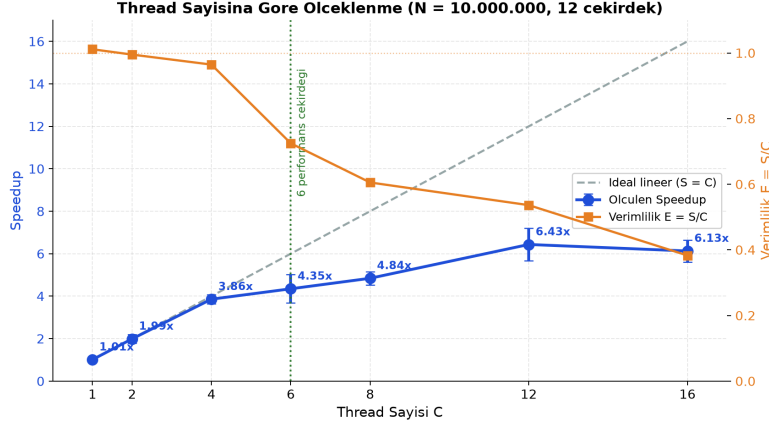
$$e = \frac{\frac{1}{S} - \frac{1}{C}}{1 - \frac{1}{C}}. \quad (7)$$

Tablo 3'te e , 4 thread'e kadar ≈ 0.01 (neredeyse ideal) iken $C \geq 6$ 'da ~ 0.08 – 0.11 'e yükselir. Bu *sabit olmayan, artan* seri pay klasik bir bellek/iletişim darboğazı değil, **heterojen çekirdek** etkisidir: M3 Pro 6 *performans* + 6 *verim* çekirdeği barındırır. İlk ~ 6 thread hızlı P-çekirdeklerine yerleşince verimlilik ≈ 1 kalır ($E_4 = \%96$); sonraki thread'ler yavaş E-çekirdeklerine düştükçe marjinal katkı azalır ve verimlilik düşer ($E_{12} = \%54$). $C = 16$ aşırı-aboneliklidir ve ek kazanç gürültü düzeyindedir.

“Embarrassingly parallel” ama neden $12\times$ değil? Çekirdek hesabı (çapraz çarpım + `atan2`) veri-bağımsızdır, dolayısıyla *embarrassingly parallel*'dir. Yine de tavan $\approx 6.3\times$ 'tir çünkü (i) 6 P-çekirdeği gerçek yüksek başarımlı taşıyor, E-çekirdekleri yavaştır; (ii) `ArrayList<Point>` referans dizisi olduğundan her erişim pointer-chasing gerektirir ve bellek bant genişliğinde yarışa yol açar. Bu, “paralel her zaman hızlıdır” yargısının neden yanlış olduğunu gösterir: **paralleleştirme, problem boyutu koordinasyon maliyetini aştıktan sonra ve mevcut fiziksel çekirdek sınırı ölçüsünde anlamlıdır.**

C (thread)	T_{seri} (ms)	T_{paralel} (ms)	S	$E = S/C$	Karp-Flatt e
1	86.35	85.39	$1.01\times$	%101	—
2	88.38	44.41	$1.99\times$	%99	0.005
4	84.44	21.89	$3.86\times$	%96	0.012
6	87.06	20.02	$4.35\times$	%72	0.076
8	83.77	17.30	$4.84\times$	%61	0.093
12	82.20	12.77	$6.43\times$	%54	0.079
16	87.79	14.33	$6.13\times$	%38	0.107

Tablo 3: Sabit $N = 10^7$ için thread sayısına göre ölçeklenme. $C = 16$ aşırı-abonelik (oversubscription); makinede yalnızca 12 çekirdek vardır.



Şekil 3: Thread sayısına göre Speedup (mavi) ve verimlilik $E = S/C$ (turuncu, sağ eksen). Verimlilik 4 thread'e kadar ≈ 1 ; 6 performans çekirdeği sınırından (yeşil çizgi) sonra düşer.

7 Sınırlamalar ve Gelecek Çalışmalar

Tek donanım: sonuçlar M3 Pro'ya özgüdür; homojen çekirdekli bir x86 sunucuda verimlilik eğrisi farklı çıkar. **Bellek düzeni:** `double[] xs, ys` gibi bitişik (SoA) bir düzen pointer-chasing'i kaldırarak hem seri hem paralel başarımları artırabilir. **Ölçüm gürültüsü:** standart sapma rapor ve figürlerde verildi; küçük N 'de alt-milisaniye süreler göreceli gürültüyü artırır. **Karşılaştırma:** `Stream.parallel()` / `ForkJoinPool` kıyası ve Java Vector API (SIMD) ile mikro-paralelizm gelecek çalışmadır.

8 Sonuç

Çapraz çarpım işareti $ve \pm 2\pi$ toplam-dönüş koşuluna dayanan algoritma, ardışık üçlülerin bağımsızlığı sayesinde paralelleştirmeye son derece uygundur ve pentagram gibi kendiyle kesişen girdileri de doğru reddeder. Paylaşılan `ExecutorService` + kilitsiz `decide()` reduction'ı ile temiz bir paralel mimari kuruldu. Apple M3 Pro üzerinde $N = 10^7$ için **$6.33\times$** hızlanma ölçüldü; verimlilik 6 performans çekirdeği sınırına kadar ≈ 1 kalıp sonra düştü. Bulgular Amdahl ve Karp-Flatt metrikleriyle tutarlıdır ve hızlanmanın hem problem boyutuna hem de fiziksel çekirdek yapısına bağlı olduğunu gösterir.

Çalıştırma. `javac -d out src/*.java ; java -Xmx4g -cp out Main` (doğruluk testlerini koşar ve süreleri `results.csv` dosyasına yazar). Kaynak kodu: <https://github.com/YunusAlim/paralelProgramlama>.